

AsynFormer: Transformer capturing asynchronous cross-variate dependencies for efficient multivariate time series forecasting

Peng He ^a, Yanglei Gan ^{a,*}, Yuxiang Liu ^a, Run Lin ^a, Guanyu Zhou ^a, Jun Xie ^b, Yao Liu ^c, Qiao Liu ^{a,*}

^a School of Computer Science and Engineering, University of Electronic Science and Technology of China, Chengdu, Sichuan, 611731, China

^b China Electronics Technology Group Corporation 54th Research Institute, Shijiazhuang, Hebei, 050081, China

^c School of Information and Software Engineering, University of Electronic Science and Technology of China, Chengdu, Sichuan, 610054, China

ARTICLE INFO

Keywords:

Multivariate time series forecasting

Asynchronous dependencies

Variable-wise modeling

ABSTRACT

Multivariate Time Series Forecasting (MTSF) is fundamental to a wide array of domains where operational and strategic decisions depend on accurate predictions. Effective forecasting in these domains is crucial, demanding precise modeling of complex patterns, including cross-variate dependencies and intra-series dynamics. Distinctive trend characteristics in each time series pose challenges; while existing architectures attempt to partition time series into patches and apply self-attention on these patches. However, they either neglect asynchronous cross-variate interactions (e.g., lead-lag effects) or incur quadratic complexity as the number of variables grows. To bridge this gap, we propose a transformer-based model AsynFormer to capture intra-series and cross-variate correlations simultaneously. Additionally, we design a variable-wise proxy module which reduces the complexity and ensures the model's scalability for high-dimensional datasets. We conducted extensive experiments on long-term and short-term forecasting tasks across eleven real-world datasets. For long-term forecasting, AsynFormer achieves state-of-the-art performance in over 70% of cases (49/70) against baseline models. For short-term forecasting, it improves upon the best baseline (AMD), with a 3.59% reduction in Mean Squared Error (MSE) and a 5.04% reduction in Mean Absolute Error (MAE). Ultimately, AsynFormer provides a scalable and robust framework, demonstrating that it is possible to model complex asynchronous dependencies without sacrificing favorable run-time efficiency.

1. Introduction

Multivariate Time Series Forecasting (MTSF) is an essential function for various domains, such as finance [1,2], intelligent transportation [3,4], meteorology [5,6], and energy systems [7,8], where reliable predictions drive both operational efficiency and strategic planning. Accurate MTSF depends on capturing two primary patterns [9–12] inherent in multivariate time series: *cross-variate dependencies* and *intra-series dynamics*. The former describes the correlations and causal influences among different variables, while the latter captures both long-term trends and short-term fluctuations within each individual series.

Patch-based methods address intra-series dynamics by dividing each variable's sequence into temporal segments [13,14], thereby modeling within-series fluctuations in a fine-grained fashion. However, this segmentation inherently overlooks cross-variate interactions. In contrast, iTransformer [10] encodes each variable's entire temporal trajec-

tory into a single token and applies variable-wise self-attention to capture cross-variate dependencies, at the expense of fine-grained temporal modeling. Recent architectures [9,11] integrate both variable-wise and patch-wise attention mechanisms to jointly learn cross-variate correlations and intra-series fluctuations; nevertheless, their computational cost can become prohibitive when dealing with a large number of series variables [15].

Moreover, existing methods assume that cross-variable influences unfold synchronously, either by interleaving or parallelizing variable-wise and patch-wise attention modules [16–18]. However, in real-world systems, multivariate interactions are frequently asynchronous and delayed. A prominent example is the lead-lag correlation within the automotive supply chain, as illustrated in Fig. 1: an upstream shortage in raw materials first leads to a reduction in battery production, which subsequently results in a lagged decline in downstream automotive sales. Similarly, in diabetes management, a noticeable drop in blood glucose is

* Corresponding author.

E-mail addresses: hepenglk@std.uestc.edu.cn (P. He), yangleigan@std.uestc.edu.cn (Y. Gan), 202321081212@std.uestc.edu.cn (Y. Liu), runlin@std.uestc.edu.cn (R. Lin), 202522080824@std.uestc.edu.cn (G. Zhou), 913247579@qq.com (J. Xie), liuyao@uestc.edu.cn (Y. Liu), qliu@uestc.edu.cn (Q. Liu).

<https://doi.org/10.1016/j.knosys.2026.115557>

Received 5 November 2025; Received in revised form 2 February 2026; Accepted 12 February 2026

Available online 19 February 2026

0950-7051/© 2026 Published by Elsevier B.V.

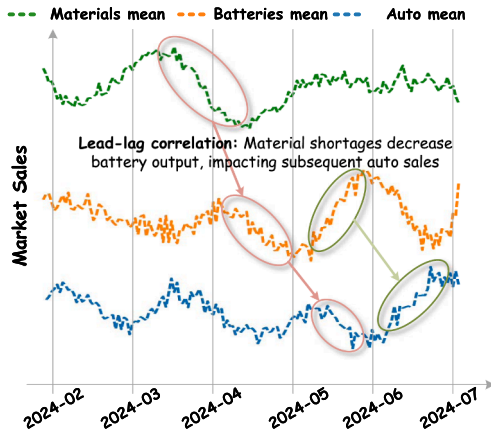


Fig. 1. Lead-Lag Correlation in the Automotive Sector: A shortage in raw material supply leads to reduced battery production, subsequently resulting in a decline in vehicle sales.

observed approximately one hour after subcutaneous insulin injection due to absorption dynamics [19]. To further elucidate the underlying mechanisms, we plot the patch-level Pearson correlation heatmaps of the variables for the Weather, ETTh1/h2/m1 datasets.¹ As illustrated in Fig. 2, the diagonal regions in sub-figures (a-d) exhibit distinct band-like structures, indicating consistent synchronous coupling among these variables. However, the off-diagonal regions of the subfigures (e-h) are characterized by scattered and offset clusters, rather than being concentrated along the diagonal, which reveals the existence of complex and non-uniform lagged correlations across patches. These observations highlight the insufficiency of purely synchronous patch-wise attention mechanisms. Therefore, a new architectural paradigm is required that can effectively and efficiently model these critical lead-lag relationships to yield significant improvements in forecasting accuracy.

Going beyond the above mentioned designs, we further exploit asynchronous cross-variate correlations and intra-series dynamics inherent in multivariate time series. Technically, we propose **AsynFormer**, a transformer-based framework that simultaneously captures cross-variate dependencies at Asynchronous time points and local temporal patterns within each variate through patch-wise modeling. To ensure scalability in high-dimensional settings, we introduce a variable-wise proxy module built from variable-specific learnable tokens that efficiently compress global temporal context, thereby reducing the quadratic complexity of self-attention by decoupling it from the sequence length. Additionally, we incorporate a frequency-domain regularization loss that explicitly aligns the spectral characteristics between predicted and ground-truth sequences, enhancing temporal coherence and robustness across extended forecasting horizons. Our main contributions are three-fold:

- We propose **AsynFormer**, a dual-stream Transformer framework that couples an asynchronous variable-wise attention branch for lagged dependencies with a patch-wise temporal branch, simultaneously capturing cross-variate interactions and fine-grained intra-series dynamics.
- We design a variable-wise proxy module that replaces full pairwise cross-attention among all patches of variables with attention over a small set of learnable proxy tokens to preserve computational efficiency.
- Extensive experiments on eleven datasets indicate that AsynFormer achieves consistent state-of-the-art performance in both long-term and short-term forecasting tasks with superior efficiency on a wide range of benchmarks.

¹ Computational details can be found in Section 3.

2. Related works

The surge of Transformer-based architectures in natural language processing [20] and computer vision [21] has catalyzed their application to time series forecasting. Unlike Temporal Convolutional Networks (TCNs) [22,23] and Recurrent Neural Networks (RNNs) [24,25] models, Transformers natively support long-range dependencies and parallel processing of sequence data, offering both modeling power and scalability. To tailor Transformers for forecasting, recent variants have advanced four key areas: novel attention mechanisms that enhance dependency discovery [26,27], bespoke temporal positional encodings that improve temporal awareness [28,29], sparse and linearized attention schemes that reduce quadratic complexity [30,31], and hierarchical or multi-scale designs that capture patterns at different temporal resolutions [14,32]. Based on our systematic review of Transformer-based forecasters, we categorize existing advances into two primary classes: intra-series temporal modeling and cross-variate dependency modeling.

Intra-series Temporal Modeling. Capturing the intrinsic temporal variations within each series is critical for accurate forecasting [11,33]. Early Transformer adaptations such as Reformer [30] and Informer [27] represent each time step as a multivariate token and apply standard self-attention to learn dependencies across all timestamps. However, these models can suffer from distribution shifts over long horizons, leading to degraded performance. To mitigate this issue, recent methods partition the original sequence into consecutive patches, each summarizing a local window of time, and apply self-attention mechanisms [13,32,34] or other deep learning networks [35] to these patch embeddings. By focusing on aggregated temporal segments rather than individual steps, patch-based approaches improve robustness to noise and distributional shift while preserving fine-grained dynamics within each patch. Nonetheless, by treating each variable channel independently within patches, these models overlook cross-variate correlations [11], which impedes performance in multivariate forecasting scenarios.

Furthermore, beyond patch-based architectures, recent advancements have explored decomposition-based strategies to simplify the forecasting task by breaking down multivariate time series into simpler subsequences [36–40]. The core philosophy of these methods is to decompose complex, non-stationary signals into a finite set of *Intrinsic Mode Functions* (IMFs). Each IMF captures local characteristic signals at specific time scales, allowing the model to effectively isolate and handle complex temporal patterns [40,41]. Representative works have demonstrated the efficacy of this paradigm. For instance, EMD-SVR [36] employs Bivariate Empirical Mode Decomposition (BEMD) combined with Support Vector Regression (SVR) to model the lower and upper bounds of hourly electricity demand, thereby capturing potential correlations between series. Similarly, MEMD-TCN [39] proposes a hybrid stock index forecasting model based on Multivariate Empirical Mode Decomposition (MEMD) and TCN, where final predictions are derived by aggregating the results from all decomposed subsequences. Notably, the recent MA-EMD [40] addresses the critical issue of “mode mixing” in multivariate contexts by utilizing Aligned Empirical Mode Decomposition. By aligning subsequences based on intrinsic frequency scales prior to forecasting, MA-EMD validates the significant benefit of physically aligning frequency components for capturing lead-lag relationships. However, such decomposition-based methods typically operate as decoupled preprocessing modules. This separation may prevent the full utilization of the end-to-end learning capabilities of deep neural networks, potentially limiting the model’s efficiency in adaptively capturing rapid fluctuations and high-frequency details compared to integrated optimization strategies.

Cross-variate Dependency Modeling. Effective multivariate forecasting necessitates the rigorous modeling of dynamic variable interactions. While standard Transformer-based methods often treat variates independently, recent advancements have introduced specialized

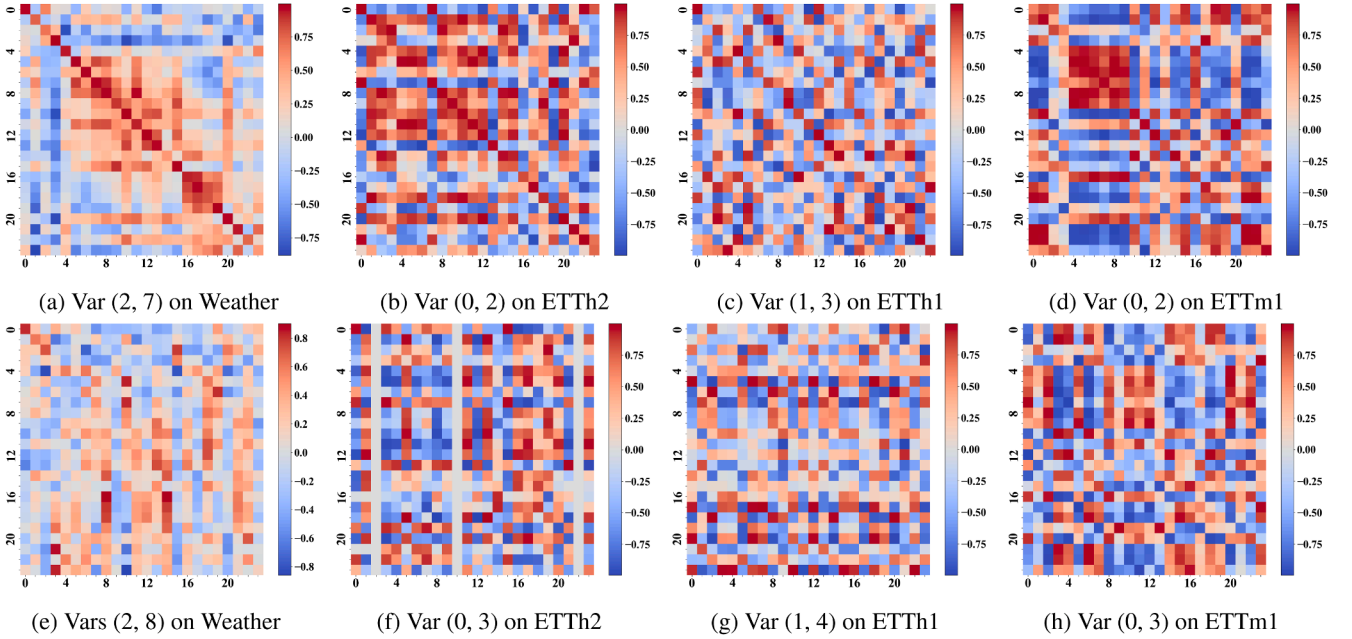


Fig. 2. Visualization of patch-wise correlation across different timestamps from different variates. 'Var(2, 7)' means variable 2 is the x-axis and variable 7 is the y-axis. (a) Var (2, 7) on Weather. (b) Var (0, 2) on ETTh2. (c) Var (1, 3) on ETTh1. (d) Var (0, 2) on ETTm1. (e) Vars (2, 8) on Weather. (f) Var (0, 3) on ETTh2. (g) Var (1, 4) on ETTh1. (h) Var (0, 3) on ETTm1.

architectures to capture cross-dimension dependencies. Crossformer [9] employs a Two-Stage Attention layer to sequentially capture cross-time and cross-dimension dependencies; however, its rigid Dimension-Segment-Wise embedding implies a synchronous alignment assumption, often leading to error accumulation in non-aligned signals. Similarly, while CARD [42] utilizes multi-scale fusion to refine representations, it incurs high computational overhead and primarily focuses on aligned correlations, overlooking asynchronous shifts. Moving towards variate-centric designs, iTransformer [10] aggregates the entire sequence of each variate into a single token to capture global interactions. Building on this, Leddam [11] parallelizes computations via a dual attention block. However, performing channel-wise attention directly on full-sequence feature maps results in computational costs proportional to the sequence length and implicitly assumes synchronous interactions. Furthermore, TimeXer [43] leverages series-level proxies to ingest exogenous information via an Endogenous-Exogenous Cross-Attention mechanism. Yet, by treating the target series as a single query attending to a pooled set of exogenous keys, it tends to homogenize the distinct temporal influences of heterogeneous source variables, limiting its effectiveness in MTSF tasks.

Beyond standard attention adaptations, other approaches focus on explicitly modeling physical correlations and alignments. LIFT [44] leverages locally stationary lead-lag correlations, allowing lagging variables to incorporate information from leading indicators to improve accuracy. To capture structural dependencies, FC-STGNN [45] proposes a fully-connected spatial-temporal graph to explicitly model correlations across all timestamps. More recently, MRLCD-A [46] introduces a multivariate rolling-layer correlation technique that computes distances between variables across different windows, performing forecasting via an alignment mechanism. Although these architectures advance dependency modeling, they either suffer from quadratic computational complexity or struggle to effectively align cross-variate signals within a unified end-to-end framework. In contrast, AsynFormer integrates intra-series temporal dynamics and cross-variate dependency modeling into a cohesive framework. By utilizing efficient variable-wise proxies, it captures the asynchronous lead-lag interactions between variables across different temporal patches without the prohibitive costs of full-sequence attention.

3. Preliminaries

Problem Definition. Given an ordered sequence of T historical observations, denoted as $\mathbf{X} = \{\mathbf{x}_{T-L:T}^c\}_{c=1}^C$, where L denotes the look-back window and C denotes the number of channels. The objective is to construct a predictive model $f(\cdot)$, this model should learn the underlying patterns and uncertainties inherent in the sequence $\mathbf{X}$ to estimate the future H unknown values, thereby generating the forecast sequence $\hat{\mathbf{Y}} = \{\hat{\mathbf{x}}_{T:T+H}^c\}_{c=1}^C$. The goal of forecasting is to ensure that each predicted value $\hat{\mathbf{Y}}$ closely approximates the corresponding future ground-truth observation $\mathbf{Y} = \{\mathbf{x}_{T:T+H}^c\}_{c=1}^C$. Prediction accuracy is typically quantified by minimizing a specific loss function $\mathcal{L}(\mathbf{Y}, \hat{\mathbf{Y}})$, such as Mean Squared Error or Mean Absolute Error.

Correlation Coefficient in Series. To verify the existence of distinct intra-series and cross-variable dependencies within real-world data, we employ the correlation coefficient to quantify the relationship between $\mathbf{X}_{t:t+S}^{(i)}$ and $\mathbf{X}_{t':t'+S}^{(j)}$, where $\mathbf{X}$ is the input sequence, and i, j are different variables. This calculation is formalized as:

$$\mathbf{R}^{(i,j)}(t, t', S) = \frac{\text{Cov}(\mathbf{X}_{t:t+S}^{(i)}, \mathbf{X}_{t':t'+S}^{(j)})}{\sigma^{(i)} \sigma^{(j)}} = \frac{1}{S} \sum_{s=0}^S \frac{\mathbf{X}_{t+s}^{(i)} - \mu^{(i)}}{\sigma^{(i)}} \cdot \frac{\mathbf{X}_{t'+s}^{(j)} - \mu^{(j)}}{\sigma^{(j)}}, \quad (1)$$

where $\mu^{(\cdot)}$ and $\sigma^{(\cdot)}$ are the mean and standard deviation of corresponding time series patches and $\mathbf{R}^{(i,j)}$ is the correlation coefficient between $\mathbf{X}_{t:t+S}^{(i)}$ and $\mathbf{X}_{t':t'+S}^{(j)}$.

4. Methodology

4.1. Patch embedding

To precisely capture evolving patterns both within and across time series, AsynFormer employs a patch-wise embedding strategy, where the input sequence $\mathbf{X}$ is partitioned into non-overlapping patches, as shown in Fig. 3. Each patch is subsequently embedded into a token incorporating positional information. Furthermore, to capture intra-series dynamics, we randomly initialize *series-wise proxies* for each variate. Moreover,

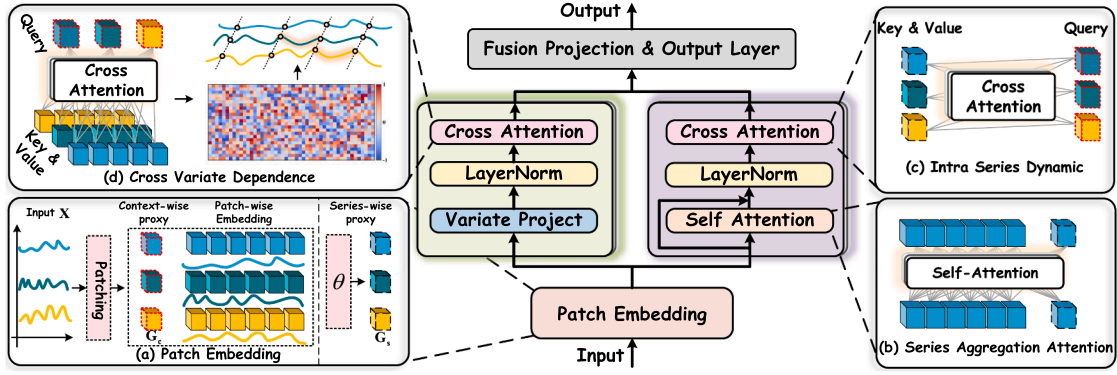


Fig. 3. The overall framework of AsynFormer, which employs a dual-stream Asynchronous learning architecture based on Transformer to capture intra-series dynamics and cross-variate dependencies, wherein: (a) Patch embedding module partitions input series into non-overlapping patch, concurrently generating both *context-wise proxy* and *series-wise proxy*; (b) Series aggregation module utilizes self-attention to empower *series-wise proxy* with the capacity to learn intra-series fluctuations; (c) Intra-series module integrates temporal dependencies through cross-attention mechanism; (d) Cross-variate module enables direct interaction between *context-wise proxy* across variables, thereby modeling asynchronous correlations.

we encode the entire series to derive *context-wise proxies*, defined as follows:

$$\begin{aligned} \{\mathbf{p}_1, \mathbf{p}_2, \dots, \mathbf{p}_N\} &= \text{Patching}(\mathbf{X}), \\ \mathbf{P} &= \text{Embedding}(\mathbf{p}_1, \mathbf{p}_2, \dots, \mathbf{p}_N; \text{POS}), \\ \mathbf{G}_s &= \text{Learnable}_{\text{Series}}(\mathbf{X}), \quad \mathbf{G}_c = \text{Learnable}_{\text{Context}}(\mathbf{X}), \end{aligned} \quad (2)$$

where $N = \lfloor \frac{L}{S} \rfloor$, derived from the patch length S and look-back window L . POS denotes the positional encoding, and $\mathbf{P} \in \mathbb{R}^{B \times C \times N \times D}$ represents the patch-level tokens incorporating POS , where B is batch size, C is number of variables and D is hidden dimension. $\text{Learnable}_{\text{Series}}(\cdot)$ represents a learnable, series-wise proxy generator parameterized by θ , serving as a global token for the series, and $\text{Learnable}_{\text{Context}}(\cdot)$ denotes a contextual proxy generator that directly encodes the entire sequence $\mathbf{X}$ into a single vector (i.e., it sets the patch length S to L and applies one linear layer for learning). $\mathbf{G}_s, \mathbf{G}_c \in \mathbb{R}^{B \times C \times 1 \times D}$ denote series-wise proxies and context-wise proxies, respectively.

4.2. Dual-stream network

4.2.1. Intra-series dynamics learning

In order to capture the temporal dependencies within the intra-series, this phase focuses on fine-grained modeling at the temporal perspective. Firstly, the patch embedding $\mathbf{P}$ are treated as local transient patterns reflecting fine-grained temporal dynamics, while $\mathbf{G}_s$ aggregates these patch embedding via self-attention. Subsequently, the context proxies $\mathbf{G}_c$ Queries this aggregated proxies $\mathbf{G}_s$ through a cross-attention to capture intra-series evolutionary characteristics.

Series Aggregation Attention. Within our experimental framework, self-attention mechanisms are leveraged to endow series-wise proxies with the capacity to learn intra-series fluctuations, thus enabling granular modeling of temporal dynamics at fine-grained resolutions, as described:

$$\hat{\mathbf{G}}_s = \text{LN}(\text{Self-Attention}(\mathbf{P}, \mathbf{G}_s)), \quad (3)$$

where LN denotes Layer Normalization. $\hat{\mathbf{G}}_s \in \mathbb{R}^{B \times C \times 1 \times D}$ denotes the fine-grained temporal dynamic aggregation of patch signals.

Intra Series Dynamic. To enhance contextual guidance signals and integrate intra-series temporal dependencies, we introduce cross-attention between aggregated series proxies $\hat{\mathbf{G}}_s$ and contextual proxies $\mathbf{G}_c$. This mechanism leverages temporal priors to refine the aggregation process while assimilating temporal patterns within intra-series:

$$\mathbf{O}_{\text{intra}} = \text{Cross-Attention}(\hat{\mathbf{G}}_s, \mathbf{G}_c), \quad (4)$$

where $\mathbf{G}_c$ denotes as *Queries* and $\mathbf{O}_{\text{intra}} \in \mathbb{R}^{B \times C \times 1 \times D}$.

4.2.2. Cross-variate dependencies learning

To model asynchronous dependencies among variables, we first subject contextual proxies $\mathbf{G}_c$ and patch embeddings $\mathbf{P}$ to dimensional transformations. These transformed representations are subsequently assigned as *Queries* and *Keys & Values* within a cross-attention mechanism, facilitating exhaustive cross-variable interactions, as described:

$$\begin{aligned} \hat{\mathbf{P}}, \hat{\mathbf{G}}_c &= \text{Project}(\mathbf{P}, \mathbf{G}_c), \\ \hat{\mathbf{O}}_{\text{inter}} &= \text{Cross-Attention}(\text{LN}(\hat{\mathbf{P}}), \hat{\mathbf{G}}_c), \\ \mathbf{O}_{\text{inter}} &= \text{Reproject}(\hat{\mathbf{O}}_{\text{inter}}), \end{aligned} \quad (5)$$

where $\text{Project}(\cdot)$ reshapes the dimensions using $M = C \times N$, subsequently yielding $\hat{\mathbf{P}}, \hat{\mathbf{G}}_c \in \mathbb{R}^{B \times M \times D}$. $\text{Reproject}(\cdot)$ denotes the inverse restoration operation. Finally the output is represented as $\mathbf{O}_{\text{inter}} \in \mathbb{R}^{B \times C \times 1 \times D}$.

4.3. Fusion projection and output layer

Ultimately, we consolidate all patch embeddings $\mathbf{P}$, intra-series learning $\mathbf{O}_{\text{intra}}$, and cross-variate learning $\mathbf{O}_{\text{inter}}$ into a unified representation. This fused representation is then processed through a Feed-Forward Network to obtain the final output representation, defined as follows:

$$\begin{aligned} \mathbf{O}_F &= \text{Fusion}(\mathbf{P}, \mathbf{O}_{\text{intra}}, \mathbf{O}_{\text{inter}}), \\ \mathbf{O}_{\text{out}} &= \text{Feed-Forward}(\mathbf{O}_F), \\ \hat{\mathbf{Y}} &= \text{Flattening-Head}(\mathbf{O}_{\text{out}}), \end{aligned} \quad (6)$$

where $\mathbf{O}_F \in \mathbb{R}^{B \times C \times (N+2) \times D}$, and Feed-Forward contains residual structure, layer normalization and dropout technique. Flattening-Head includes patch mapping and linear projection. $\hat{\mathbf{Y}} \in \mathbb{R}^{B \times H \times C}$ and H is target window length.

4.4. Training objective

To augment the supervision signals during model training, we adopt a composite loss function proposed in work [34], which comprises two components: Mean Absolute Error (MAE) and Frequency-domain loss.

Prediction Loss. For the forecaster, MAE loss $\mathcal{L}_M$ is used to measure the variance between predicted values and ground-truth $\mathbf{Y}$, defined as follows:

$$\mathcal{L}_M = \frac{1}{H} \sum_{h=1}^H |\hat{\mathbf{Y}} - \mathbf{Y}|, \quad (7)$$

Algorithm 1 Training procedure of AsynFormer.

Input: Multivariate time series $\mathbf{X} \in \mathbb{R}^{B \times L \times C}$, Ground truth $\mathbf{Y} \in \mathbb{R}^{B \times H \times C}$, hyperparameter α .

Output: Optimized model parameters θ^* .

```

1: while not converged do
2:   Step 1: Patching and Proxies Generation (Eq. (2))
3:    $\{\mathbf{p}_1, \dots, \mathbf{p}_N\} = \text{Patching}(\mathbf{X})$   $\triangleright$  Partition input
4:    $\mathbf{P} = \text{Embedding}(\{\mathbf{p}_i\}_{i=1}^N; \text{POS})$   $\triangleright$  Embed with Pos.
5:    $\mathbf{G}_s = \text{Learnable}_{\text{Series}}(\mathbf{X})$   $\triangleright$  Series-wise proxies
6:    $\mathbf{G}_c = \text{Learnable}_{\text{Context}}(\mathbf{X})$   $\triangleright$  Context-wise proxies
7:
8:   Step 2: Intra-series Dynamics Learning
9:    $\hat{\mathbf{G}}_s = \text{Self-Att.}(\mathbf{P}, \mathbf{G}_s)$   $\triangleright$  Series agg. via Eq. (3)
10:   $\mathbf{O}_{\text{intra}} = \text{Cross-Att.}(\hat{\mathbf{G}}_s, \mathbf{G}_c)$   $\triangleright$  Refine via Eq. (4)
11:
12:  Step 3: Cross-variate Dependencies (Eq. (5))
13:   $\hat{\mathbf{P}}, \hat{\mathbf{G}}_c = \text{Project}(\mathbf{P}, \mathbf{G}_c)$   $\triangleright$  Transformation
14:   $\hat{\mathbf{O}}_{\text{inter}} = \text{Cross-Att.}(\text{LN}(\hat{\mathbf{P}}), \hat{\mathbf{G}}_c)$   $\triangleright$  Async. modeling
15:   $\mathbf{O}_{\text{inter}} = \text{Reproject}(\hat{\mathbf{O}}_{\text{inter}})$   $\triangleright$  Restore dim.
16:
17:  Step 4: Fusion and Output (Eq. (6))
18:   $\mathbf{O}_F = \text{Fusion}(\mathbf{P}, \mathbf{O}_{\text{intra}}, \mathbf{O}_{\text{inter}})$   $\triangleright$  Unified output
19:   $\mathbf{O}_{\text{out}} = \text{Feed-Forward}(\mathbf{O}_F)$ 
20:   $\hat{\mathbf{Y}} = \text{Flattening-Head}(\mathbf{O}_{\text{out}})$   $\triangleright$  Final prediction
21:
22:  Step 5: Loss Calculation and Update
23:   $\mathcal{L}_M = \frac{1}{H} \sum |\hat{\mathbf{Y}} - \mathbf{Y}|$   $\triangleright$  MAE loss via Eq. (7)
24:   $\mathcal{L}_F = \frac{1}{H} \sum |\mathcal{F}(\hat{\mathbf{Y}}) - \mathcal{F}(\mathbf{Y})|$   $\triangleright$  Freq. loss via Eq. (8)
25:   $\mathcal{L}_S = (1 - \alpha) \cdot \mathcal{L}_M + \alpha \cdot \mathcal{L}_F$   $\triangleright$  Total target via Eq. (9)
26:   $\theta \leftarrow \theta - \eta \nabla_{\theta} \mathcal{L}_S$   $\triangleright$  Update parameters
27:
28: end while
29: return  $\theta^*$ 

```

Frequency Domain Loss. We first apply the Fast Fourier Transform (FFT) to project both the predicted series $\hat{\mathbf{Y}}$ and target series $\mathbf{Y}$ into the frequency domain. The MAE is then computed between their magnitude spectra to quantify spectral discrepancies. This process is defined as follows:

$$\mathcal{L}_F = \frac{1}{H} \sum_{h=1}^H |\mathcal{F}(\hat{\mathbf{Y}}) - \mathcal{F}(\mathbf{Y})|, \quad (8)$$

where $\mathcal{F}$ is FFT, and our training objective is denoted as:

$$\mathcal{L}_S = (1 - \alpha) \cdot \mathcal{L}_M + \alpha \cdot \mathcal{L}_F, \quad (9)$$

where α is a hyperparameter that control the strength of the frequency-domain loss $\mathcal{L}_F$ and MAE loss $\mathcal{L}_M$.

Finally, we summarize the training process of AsynFormer, and the results are shown in [Algorithm 1](#).

4.5. Complexity analysis

To evaluate computational efficiency, we compare AsynFormer with standard multivariate Transformers. Let N denote the number of patches per variate. Standard approaches utilizing global self-attention across all flattened patches suffer from a complexity of $\mathcal{O}(C^2 N^2) \propto \mathcal{O}(C^2 L^2)$, which creates a quadratic bottleneck with respect to sequence length L . In contrast, AsynFormer decouples this process into two efficient stages: (1) parallel intra-series modeling with a complexity of $\mathcal{O}(CN^2)$, and (2) proxy-based cross-variate modeling with a complexity of $\mathcal{O}(C^2)$. Crucially, the cross-variate complexity is reduced from $\mathcal{O}(C^2 L^2)$ to $\mathcal{O}(C^2)$, effectively decoupling channel dependency costs from the look-back window. This design ensures that AsynFormer remains highly scalable for MTSF tasks.

5. Experiments

5.1. Experimental setup

Datasets. We comprehensively evaluated the accuracy and efficiency of AsynFormer on eleven public benchmarks. These include seven datasets for long-term forecasting: ETTh1, ETTh2, ETTm1, ETTm2, Exchange, Traffic, and Weather. Additionally, we utilized four traffic network datasets for short-term forecasting: PeMS03, PeMS04, PeMS07, and PeMS08. Comprehensive dataset specifications are detailed in the [Table 1](#). The specifications of these datasets are detailed as follows:

- **ETT** (Electricity Transformer Temperature) [27] is a crucial benchmark for real-world time series forecasting. This dataset contains key operational metrics, including oil temperature and power load, collected from two separate electricity transformers in China over a two-year period from 2016 to 2018. To accommodate diverse analytical requirements, the data is provided at two distinct temporal granularities: hourly (denoted by the suffix h) and every 15 min (denoted by m). This structure yields four canonical subsets widely used for evaluation: ETTh1, ETTh2, ETTm1 and ETTm2.
- **Traffic** [26] captures the hourly road occupancy rates, a key indicator of traffic congestion and flow. The data was collected from a network of 862 inductive loop sensors strategically deployed across the freeways of the San Francisco Bay Area. Spanning from 2015 to 2016, it provides a detailed and large-scale multivariate time series for modeling complex spatio-temporal traffic patterns.
- **Weather** [26] offers a comprehensive meteorological record collected throughout 2020 across Germany. It comprises 21 distinct atmospheric variables, including but not limited to air temperature, humidity, visibility, and wind speed, recorded at a fine-grained 10 min interval. The dataset's multi-faceted nature makes it an ideal testbed for models designed to capture intricate inter-variable dependencies in environmental systems.
- **Exchange** [47] is a widely recognized financial dataset that documents the daily closing foreign exchange rates of eight major economies—Australia, Switzerland, China, the United Kingdom, Canada, Japan, Singapore, and New Zealand—relative to the US Dollar. The dataset covers an extensive historical period from 1990 to 2016, making it particularly suitable for analyzing long-term economic trends and volatility.
- **PeMS** (California Performance Measurement System) [22] is a collection of large-scale traffic datasets sourced from the Caltrans PeMS. This collection includes several standard benchmarks, such as PeMS03, PeMS04, PeMS07 and PeMS08, which correspond to different districts in California. The data, which includes metrics like traffic flow and speed, is aggregated into high-resolution 5 min intervals, providing a rich source for fine-grained traffic prediction research.

Evaluation Metrics. We adopted two standard metrics for time series forecasting: Mean Absolute Error (MAE) and Mean Squared Error (MSE). Given the ground-truth series $\mathbf{Y} = \{y_1, y_2, \dots, y_H\}$ and the corresponding predicted series $\hat{\mathbf{Y}} = \{\hat{y}_1, \hat{y}_2, \dots, \hat{y}_H\}$, the MSE and MAE are respectively defined as:

$$\text{MSE} = \frac{1}{H} \sum_{h=1}^H (y_h - \hat{y}_h)^2, \text{MAE} = \frac{1}{H} \sum_{h=1}^H |y_h - \hat{y}_h|. \quad (10)$$

Baselines. For long-term forecasting, we have selected a set of state-of-the-art (SOTA) baselines covering the following three categories: (i) Transformer-based methods: FSatten [48], TimeXer [43], iTransformer [10], Crossformer [9], Autoformer [26], Pyraformer [32]; (ii) Frequency-based & other methods: FreDF [34], LIFT [44], FITS [49], TexFilter [50], FreTS [51], PatchTST [13], TiDE [52]; (iii) LLMs-based methods: TimeCMA [53], LLM4TS [54], TEMPO [55], Time-LLM [56], UniTime [57], GPT4TS [58]. For short-term forecasting, we add six well

Table 1

Dataset specifications. “Dataset Size” denotes the total number of time series in (train, validation, test) partitions. “Prediction Length” indicates the future time steps to be predicted. “Frequency” specifies the sampling interval of timestamps.

Task	Dataset	Dim	Prediction Length	Dataset Size	Frequency	information
Long-term Forecasting	ETTh1	7	{96, 192, 336, 720}	(34465, 11521, 11521)	15 min	Electricity
	ETTm2	7	{96, 192, 336, 720}	(34465, 11521, 11521)	15 min	Electricity
	ETTh1	7	{96, 192, 336, 720}	(8545, 2881, 2881)	Hourly	Electricity
	ETTh2	7	{96, 192, 336, 720}	(8545, 2881, 2881)	Hourly	Electricity
	Traffic	862	{96, 192, 336, 720}	(12185, 1757, 3509)	Hourly	Transportation
	Weather	21	{96, 192, 336, 720}	(36792, 5271, 10540)	10 min	Weather
	Exchange	8	{96, 192, 336, 720}	(5120, 665, 1422)	Daily	Economy
Short-term Forecasting	PeMS03	358	12	(15617, 5135, 5135)	5 min	Transportation
	PeMS04	307	12	(10172, 3375, 3375)	5 min	Transportation
	PeMS07	883	12	(16911, 5622, 5622)	5 min	Transportation
	PeMS08	170	12	(10690, 3548, 3548)	5 min	Transportation

-performed baselines, AMD [59], FEDformer [31], TimesNet [60], MTS-Mixers [61], DLinear [62], RLinear [63].

Implementation Details. All experiments were implemented using PyTorch and executed on a server equipped with an NVIDIA A100 GPU (80 GB). To ensure comparability, we followed on previous work [43] and used the Adam optimizer [64] for training with learning rates ranging from $\{1e-4, 2e-4, 3e-4\}$. Other experimental parameters were adjusted within the following range: batch size B was $\{4, 8, 16, 32, 64, 128\}$, and patch length S was $\{4, 8, 12, 16, 32\}$. The loss balance α ranged from $\{0.15, 0.2, 0.25, 0.3, 0.35\}$. The number of dual-stream network layers (E-layers) was selected from $\{1, 2, 3, 4, 5\}$. All baseline models were reproduced using parameters consistent with their original publications or official implementations.

5.2. Overall performance

Long-term Forecasting. We conduct evaluation experiments on seven real-world datasets: ETTh1, ETTh2, ETm1, ETTm2, Weather, Exchange, and Traffic. For all experiments in this section, we fix the look-back window L at 96 uniformly and evaluate prediction horizons $H \in \{96, 192, 336, 720\}$. Table 2 presents the MSE and MAE results. AsynFormer achieved the best performance in 49 out of 70 settings and ranked second in 12 cases. This consistent superiority demonstrates AsynFormer’s robust generalization capabilities across diverse temporal and domain characteristics. Notably, TimeXer, PatchTST and Crossformer, all of which employ a patch-based Transformer also deliver strong performance, highlighting the importance of fine-grained intra-series modeling. PatchTST’s channel-independent design, however, ignores any interactions between variables, and TimeXer’s single-token variate representation cannot capture asynchronous lead-lag effects at different timestamps. Crossformer combines variable-wise and patch-wise attention in sequence may introduce additional noise which influences their long-term forecasting as their performance degrades drastically in specific datasets as the prediction horizon increases. In contrast, AsynFormer jointly models cross-variate dependencies and intra-series fluctuations via its asynchronous attention mechanism and variable-wise proxy module, achieving consistent state-of-the-art results. These findings confirm that concurrent modeling of intra-series and cross-variate patterns is essential for accurate multivariate forecasting.

Short-term Forecasting. We assessed short-term forecasting capabilities using a 96-step look-back window to predict a 12-step horizon on four traffic datasets (PeMS03, PeMS04, PeMS07 and PeMS08). As shown in Table 3, AsynFormer consistently achieves the lowest error across all datasets. By contrast, leading patch-based methods such as PatchTST and Crossformer exhibit substantial performance degradation on these highly volatile series. Their exclusive reliance on temporal patches undermines their ability to capture rapid, localized fluctuations, causing them to lose accuracy in all PeMS settings. AsynFormer overcomes this limitation by combining patch-wise self-attention with a variable-wise

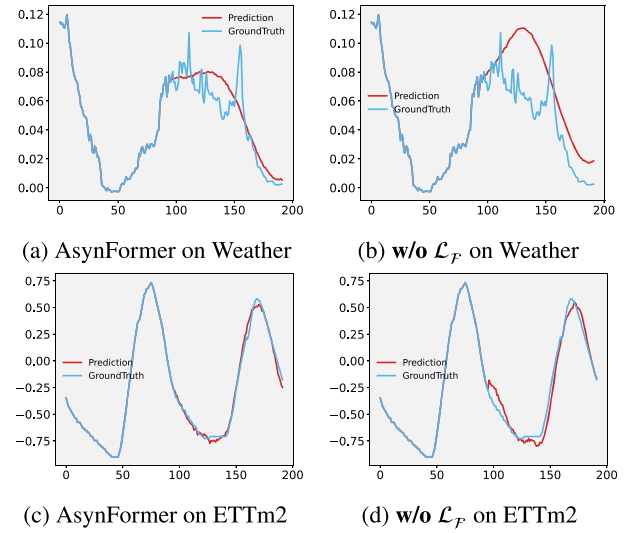


Fig. 4. Visual comparison of $w/o \mathcal{L}_F$ and AsynFormer (original) between predicted results and ground-truth. (a) AsynFormer on Weather. (b) $w/o \mathcal{L}_F$ on Weather. (c) AsynFormer on ETTm2 (d) $w/o \mathcal{L}_F$ on ETTm2.

proxy representation that aggregates global context, thereby maintaining sensitivity to abrupt changes while preserving cross-variate context.

5.3. Ablation studies

To ascertain the impact of different modules within AsynFormer, we perform ablation studies across four datasets, focusing on intra-series attention, cross-variate attention, variable-wise proxy and training objectives. The variants are defined as follows:

- **w/o Intra**, the intra-series attention module is disabled.
- **w/o Inter**, the cross-variate attention module is disabled.
- **w/o G_s** , the series-wise proxy token is disabled.
- **w/o G_c** , the context-wise proxy token is disabled.
- **w/o $\mathcal{L}_F$** , frequency-domain loss is disabled.
- **$\mathcal{L}_F + \mathcal{L}_{MSE}$** , the prediction loss is replaced by Mean Squared Error (MSE) loss.
- **$\mathcal{L}_{MSE}$** , solely rely on Mean Squared Error (MSE) loss.

Table 4 illustrates the unique impact of each module. Removing the intra-series attention module ($w/o \text{ Intra}$) increases average MSE across four datasets from 0.335 to 0.343 and MAE from 0.350 to 0.354, while disabling cross-variate attention ($w/o \text{ Intra}$) raises MSE to 0.344 and MAE to 0.354. These results confirm that both attention streams are essential: intra-series attention delivers detailed temporal modeling, and cross-variate attention captures cross-variate dependencies. Excluding

Table 3

Short-term forecasting task. The look-back window L is 96 and the prediction horizon H is 12. Details are the same as Table 2.

Model	AsynFormer	Our AMD [2025]	PatchTST [2023]	Crossformer [2023]	FEDformer [2022]	TimesNet [2023]	TiDE [2023]	DLinear [2023]	MTS-Mixers [2025]	RLinear [2023]
Metric	MSE	MAE	MSE	MAE	MSE	MAE	MSE	MAE	MSE	MAE
PeMS03	0.081	0.188	0.084	0.198	0.099	0.216	0.09	0.203	0.126	0.251
PeMS04	0.082	0.192	0.083	0.198	0.105	0.224	0.098	0.218	0.138	0.262
PeMS07	0.069	0.168	0.074	0.180	0.095	0.150	0.094	0.200	0.109	0.225
PeMS08	0.091	0.195	0.093	0.206	0.168	0.232	0.165	0.214	0.173	0.273

Table 4

Ablation results with look-back window $L = 96$ across the forecast horizons $H \in \{96, 192, 336, 720\}$ on ETTh1, ETTm1, ETTm2, and Weather datasets.

Datasets	Metrics	AsynFormer		w/o Inter		w/o Intra		w/o G_s		w/o G_c		w/o $\mathcal{L}_F$		$\mathcal{L}_F + \mathcal{L}_{MSE}$		$\mathcal{L}_{MSE}$	
		MSE	MAE	MSE	MAE	MSE	MAE	MSE	MAE	MSE	MAE	MSE	MAE	MSE	MAE	MSE	MAE
Weather	96	0.156	0.195	0.167	0.211	0.160	0.205	0.164	0.208	0.158	0.203	0.160	0.204	0.168	0.215	0.167	0.217
	192	0.202	0.240	0.223	0.250	0.217	0.247	0.218	0.246	0.215	0.244	0.215	0.244	0.220	0.254	0.221	0.255
	336	0.260	0.282	0.281	0.293	0.276	0.290	0.279	0.291	0.273	0.288	0.275	0.288	0.281	0.298	0.280	0.299
	720	0.338	0.334	0.361	0.345	0.351	0.341	0.357	0.343	0.352	0.340	0.352	0.340	0.359	0.348	0.356	0.348
	AVG	0.239	0.263	0.258	0.275	0.251	0.271	0.255	0.272	0.250	0.269	0.251	0.269	0.257	0.279	0.256	0.280
ETTh1	96	0.380	0.391	0.381	0.392	0.385	0.392	0.381	0.392	0.384	0.393	0.389	0.392	0.386	0.399	0.407	0.406
	192	0.429	0.419	0.436	0.420	0.437	0.420	0.436	0.423	0.436	0.423	0.452	0.425	0.438	0.422	0.456	0.436
	336	0.475	0.440	0.476	0.441	0.477	0.445	0.482	0.443	0.484	0.453	0.509	0.454	0.473	0.446	0.483	0.456
	720	0.466	0.460	0.471	0.461	0.483	0.473	0.495	0.478	0.475	0.463	0.492	0.469	0.478	0.470	0.594	0.535
	AVG	0.438	0.428	0.441	0.429	0.446	0.433	0.449	0.434	0.445	0.433	0.461	0.435	0.444	0.434	0.485	0.458
ETTM2	96	0.173	0.251	0.177	0.254	0.174	0.251	0.174	0.252	0.177	0.255	0.174	0.251	0.176	0.260	0.177	0.264
	192	0.236	0.295	0.241	0.297	0.242	0.297	0.239	0.296	0.242	0.297	0.239	0.195	0.242	0.301	0.244	0.305
	336	0.296	0.333	0.302	0.335	0.297	0.332	0.298	0.334	0.298	0.335	0.297	0.333	0.301	0.340	0.307	0.346
	720	0.398	0.390	0.406	0.394	0.403	0.392	0.402	0.391	0.406	0.394	0.399	0.391	0.404	0.395	0.407	0.401
	AVG	0.276	0.317	0.282	0.320	0.279	0.318	0.278	0.318	0.281	0.320	0.277	0.293	0.281	0.324	0.284	0.329
ETTM1	96	0.318	0.355	0.333	0.356	0.332	0.356	0.326	0.356	0.333	0.357	0.329	0.350	0.335	0.365	0.338	0.370
	192	0.366	0.375	0.374	0.379	0.372	0.379	0.372	0.378	0.378	0.380	0.376	0.375	0.379	0.391	0.381	0.370
	336	0.399	0.400	0.405	0.403	0.404	0.402	0.404	0.402	0.405	0.403	0.406	0.396	0.410	0.411	0.411	0.414
	720	0.466	0.437	0.471	0.439	0.472	0.438	0.472	0.439	0.471	0.439	0.479	0.435	0.475	0.439	0.477	0.444
	AVG	0.387	0.392	0.396	0.394	0.395	0.394	0.394	0.394	0.397	0.395	0.398	0.389	0.400	0.402	0.402	0.400

either the series-wise proxy token (w/o G_s) or the context-wise proxy token (w/o G_c) also compromises accuracy, underscoring how these proxies enable efficient aggregation of long-range and information across variables. We further reveal the importance of the loss function selection, ablation of the frequency-domain loss (w/o $\mathcal{L}_F$) highlights its role in aligning the model's predictions with the true spectral characteristics of the data. Without this supervision, AsynFormer struggles to maintain consistency across both high- and low-frequency components, undermining long-term accuracy [34]. We visualize the prediction results on the Weather and ETTm2 datasets in Fig. 4. As observed, the frequency domain loss ($\mathcal{L}_F$) effectively prevents the model from suffering from significant amplitude drift and over-smoothing. Without it, the predictions tend to exhibit often deviations from the ground truth, particularly in capturing peak values and rapid fluctuations, whereas the inclusion of $\mathcal{L}_F$ constrains the output to successfully follow the temporal trends and signal magnitudes. Replacing the adopted MAE loss with an MSE loss ($\mathcal{L}_F + \mathcal{L}_{MSE}$) leads to a deterioration in the MSE/MAE performance metrics. Furthermore, using a pure MSE objective function in place of the MAE and frequency loss combination ($\mathcal{L}_{MSE}$) degrades the performance even more. Collectively, these results highlight that MAE, compared to MSE, collaborates more robustly with the frequency-domain loss by mitigating the impact of large errors and maintaining overall model robustness [65].

5.4. Effectiveness under varying look-back window

In MTSF, the look-back window determines the volume of historical information available to the model. Theoretically, a larger look-back window should provide richer context and thereby improve model performance. For a fair comparison, we selected the best-performing models from our main experiments as baselines. To evaluate the effectiveness of AsynFormer with varying lengths of historical input, we configured two input lengths, $L \in \{336, 720\}$. Table 5 reports the MSE and MAE results across all prediction horizons $\{96, 192, 336, 720\}$ on three benchmark datasets. AsynFormer consistently achieves the lowest MSE and MAE across both window sizes, robustly demonstrating its ability to effectively leverage extended historical records. Notably, patch-based methods like PatchTST and TimeXer outperform other baselines, highlighting that capturing fine-grained intra-series temporal dynamics is crucial for accurate multivariate forecasting. In contrast, models such as DLinear and TimesNet show marginal improvements or even performance degradation as the input length increases. For instance, the average MSE for TimesNet on ETTm1 worsened from 0.387 to 0.400, indicating their limited capacity to integrate long-range dependencies.

Comparison with Lead-Lag Modeling Methods. It is also worth noting the performance of LIFT [44], which specifically targets channel dependence via explicit leading indicator estimation. While LIFT

Table 5

Performance across different look-back windows. With $L = \{336, 720\}$, reported results represent averages across all forecast horizons $H \in \{96, 192, 336, 720\}$. The details are the same as Table 2.

Model	AsynFormer		LIFT		TimerXer		iTransformer		PatchTST		TimesNet		Dlinear		
Metrics	MSE	MAE	MSE	MAE	MSE	MAE	MSE	MAE	MSE	MAE	MSE	MAE	MSE	MAE	
L = 336															
ETTh1	96	0.375	0.393	0.369	0.391	0.393	0.407	0.402	0.418	0.381	0.405	0.398	0.418	0.384	0.405
	192	0.410	0.414	0.407	0.416	0.415	0.416	0.450	0.449	0.442	0.446	0.447	0.449	0.430	0.442
	336	0.428	0.427	0.437	0.435	0.453	0.442	0.479	0.470	0.449	0.454	0.493	0.468	0.449	0.446
	720	0.437	0.457	0.451	0.464	0.454	0.464	0.584	0.548	0.490	0.493	0.518	0.504	0.504	0.515
	AVG	0.413	0.423	0.416	0.427	0.429	0.432	0.479	0.471	0.441	0.450	0.464	0.460	0.442	0.452
ETTm1	96	0.291	0.335	0.285	0.340	0.312	0.350	0.303	0.357	0.294	0.345	0.335	0.380	0.300	0.345
	192	0.329	0.361	0.325	0.364	0.344	0.372	0.345	0.383	0.334	0.371	0.358	0.388	0.336	0.366
	336	0.363	0.385	0.365	0.384	0.391	0.404	0.382	0.405	0.371	0.392	0.406	0.418	0.372	0.390
	720	0.419	0.415	0.422	0.423	0.426	0.424	0.443	0.439	0.421	0.419	0.449	0.443	0.427	0.423
	AVG	0.350	0.374	0.349	0.378	0.368	0.388	0.368	0.396	0.355	0.382	0.387	0.407	0.359	0.381
Weather	96	0.150	0.189	0.147	0.198	0.146	0.188	0.164	0.216	0.151	0.197	0.172	0.220	0.175	0.235
	192	0.191	0.229	0.190	0.238	0.192	0.231	0.205	0.251	0.197	0.244	0.219	0.261	0.218	0.278
	336	0.243	0.270	0.240	0.278	0.245	0.273	0.256	0.290	0.251	0.285	0.280	0.306	0.263	0.314
	720	0.312	0.323	0.318	0.332	0.315	0.325	0.326	0.338	0.321	0.335	0.365	0.359	0.324	0.362
	AVG	0.224	0.253	0.224	0.262	0.225	0.254	0.238	0.274	0.230	0.265	0.259	0.287	0.245	0.297
L = 720															
ETTh1	96	0.361	0.390	0.373	0.398	0.392	0.421	0.386	0.405	0.370	0.400	0.384	0.402	0.375	0.399
	192	0.398	0.413	0.414	0.425	0.423	0.442	0.424	0.440	0.413	0.429	0.557	0.436	0.405	0.416
	336	0.419	0.428	0.447	0.445	0.469	0.477	0.449	0.460	0.422	0.440	0.491	0.469	0.439	0.443
	720	0.424	0.447	0.469	0.478	0.484	0.497	0.495	0.487	0.447	0.468	0.521	0.500	0.472	0.490
	AVG	0.400	0.419	0.426	0.437	0.442	0.459	0.439	0.448	0.413	0.434	0.458	0.450	0.423	0.437
ETTm1	96	0.298	0.340	0.298	0.348	0.309	0.361	0.300	0.353	0.293	0.346	0.338	0.375	0.299	0.343
	192	0.325	0.363	0.343	0.374	0.335	0.371	0.345	0.382	0.333	0.370	0.371	0.387	0.335	0.365
	336	0.359	0.373	0.371	0.389	0.375	0.403	0.374	0.398	0.369	0.369	0.410	0.411	0.369	0.386
	720	0.415	0.415	0.440	0.434	0.451	0.447	0.429	0.430	0.416	0.420	0.478	0.450	0.425	0.421
	AVG	0.349	0.372	0.363	0.386	0.368	0.396	0.362	0.391	0.353	0.382	0.400	0.406	0.357	0.379
Weather	96	0.146	0.195	0.147	0.198	0.149	0.204	0.159	0.208	0.149	0.198	0.172	0.220	0.152	0.237
	192	0.191	0.239	0.190	0.239	0.210	0.257	0.200	0.248	0.194	0.241	0.219	0.261	0.220	0.282
	336	0.244	0.277	0.244	0.280	0.249	0.293	0.253	0.289	0.245	0.282	0.280	0.306	0.265	0.319
	720	0.312	0.328	0.314	0.330	0.317	0.338	0.321	0.338	0.314	0.334	0.365	0.359	0.323	0.362
	AVG	0.223	0.259	0.224	0.262	0.231	0.273	0.233	0.271	0.226	0.264	0.259	0.287	0.240	0.300

achieves the second-best results in certain cases, its reliance on explicitly estimated indicators implies a relatively static or global structural prior. This design may limit its adaptability to complex scenarios where lead-lag relationships evolve rapidly over time, such as in the ETTh1. Consequently, on ETTh1 with $L = 720$, LIFT lags behind AsynFormer (MSE 0.426 vs. 0.400). In contrast, AsynFormer utilizes asynchronous attention to capture dynamic “soft” alignments, enabling it to outperform LIFT in 17 out of 18 aggregate cases. This confirms that our implicit modeling approach offers superior robustness in handling varying temporal dynamics compared to rigid indicator estimation.

5.5. Visualization of prediction results

Prediction Visualizations. To evaluate the predictive performance of AsynFormer, we conduct a qualitative visual comparison of its forecast results against several baselines, as shown in Fig. 5. The experiments are performed on the Weather, ETTm2, and ETTh2 datasets under a unified setting of a 96-step input to predict a 96-step horizon. Overall, AsynFormer closely tracks the rapid fluctuations and gradual trends of the true time series, accurately capturing turning points and demonstrating robust time series consistency. Its performance on the ETTm2 dataset is particularly outstanding, where the predicted values almost perfectly overlap with the ground truth. In contrast, the baseline models exhibit various limitations. While TimeXer can roughly capture the overall contour of the series, it performs poorly in depicting local details. PatchTST generates smoother curves that follow the general trend

but systematically underestimates the magnitude of peaks (as seen on the Weather and ETTm2 datasets), which may reflect the limitations of its channel-independent patching strategy and restricted intra-series contextual modeling. Meanwhile, iTransformer struggles to align with either slow-moving trends or rapid fluctuations, and its predictions progressively diverge from the ground truth as the horizon extends. These qualitative findings robustly confirm our core hypothesis: jointly modeling asynchronous cross-variable dependencies and fine-grained intra-series dynamics yields more reliable and accurate long-term forecasts.

Asynchronous Cross-variate Correlation Visualization. To intuitively evaluate AsynFormer’s capability to learn **asynchronous** cross-variable dependencies, we visualize the patch-wise Pearson correlation heatmaps between pairs of variables before and after training. As illustrated in Fig. 6, we analyze the relationships on the Weather dataset (with a patch length of 8 and a setting of ‘96-input-96-prediction’) between variable 0 (x-axis) and variable 2 (y-axis), as well as variable 1 (x-axis) and variable 4 (y-axis). Before training, the correlations are predominantly concentrated along the diagonal (e.g., between patches 9, 10, 11), representing strong synchronous or near-synchronous couplings. In contrast, the off-diagonal correlations, which represent lagged relationships, are generally weak (e.g., between patches 1, 2, 3 of variable 0 and patches 7, 8, 9 of variable 4). After training, the heatmaps exhibit significant changes: correlations in many previously weak off-diagonal regions are markedly enhanced, forming new clusters of association. A similar conclusion is drawn from the analysis of vari-

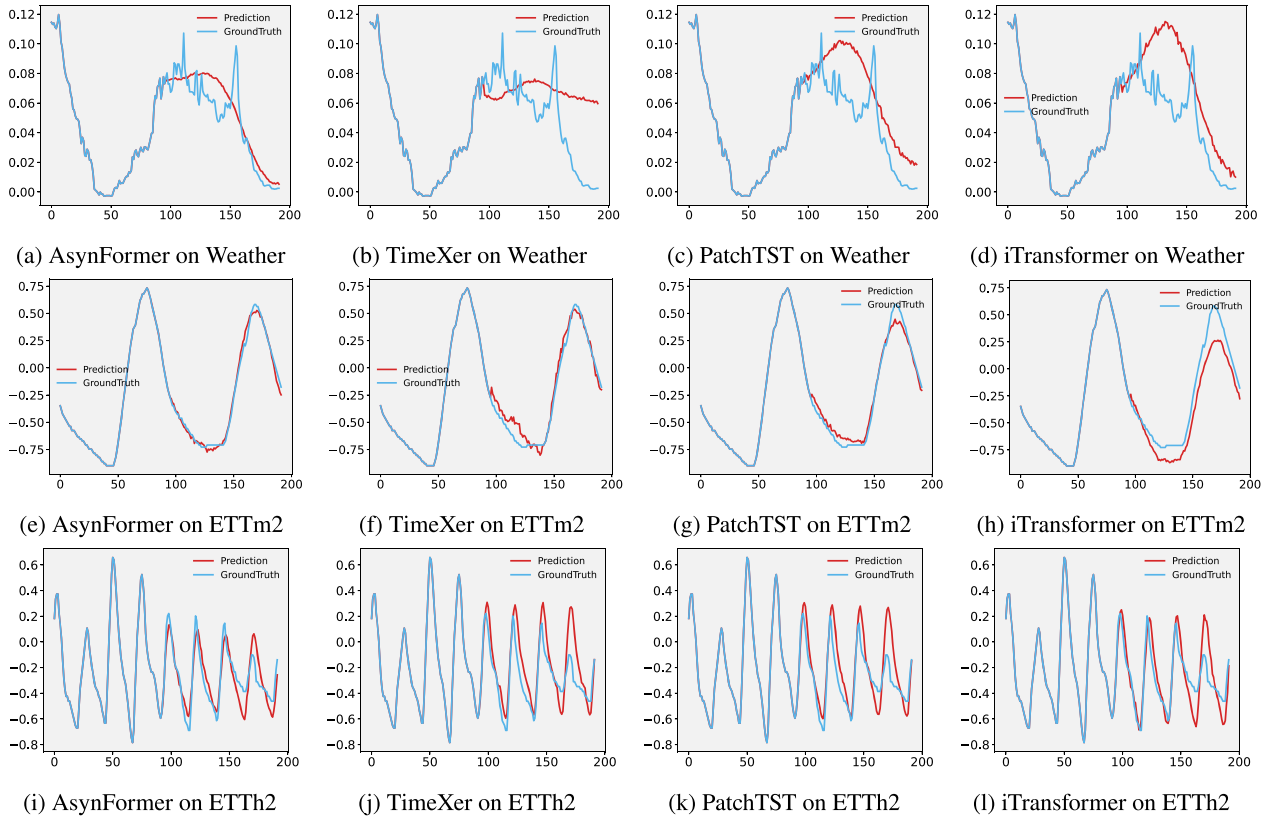


Fig. 5. Visual comparison between predicted results and ground-truth across different models on the three datasets. (a) AsynFormer on Weather. (b) TimeXer on Weather. (c) PatchTST on Weather. (d) iTransformer on Weather. (e) AsynFormer on ETTm2. (f) TimeXer on ETTm2. (g) PatchTST on ETTm2. (h) iTransformer on ETTm2. (i) AsynFormer on ETTh2. (j) TimeXer on ETTh2. (k) PatchTST on ETTh2. (l) iTransformer on ETTh2.

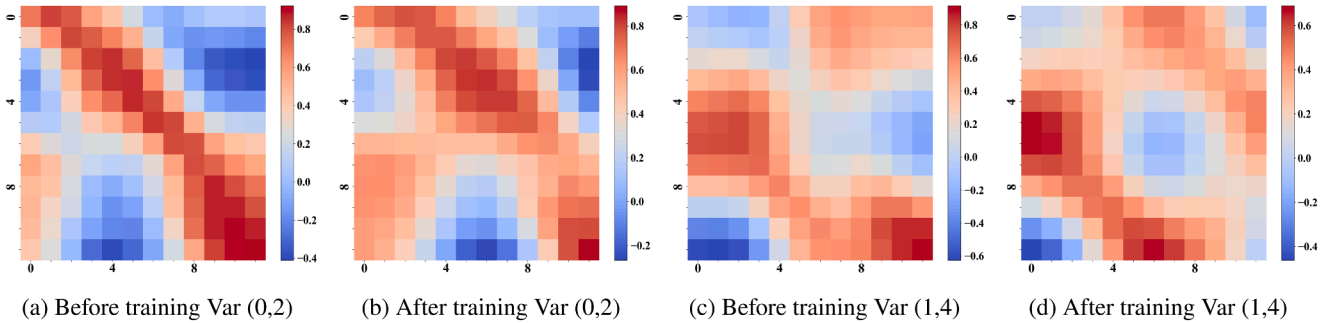


Fig. 6. Comparison of the correlation changes between variables in the weather dataset before and after training. 'Var(0,2)' indicates that variable 0 is on the x-axis and variable 2 is on the y-axis. (a) Before training Var (0,2). (b) After training Var (0,2). (c) Before training Var (1,4). (d) After training Var (1,4).

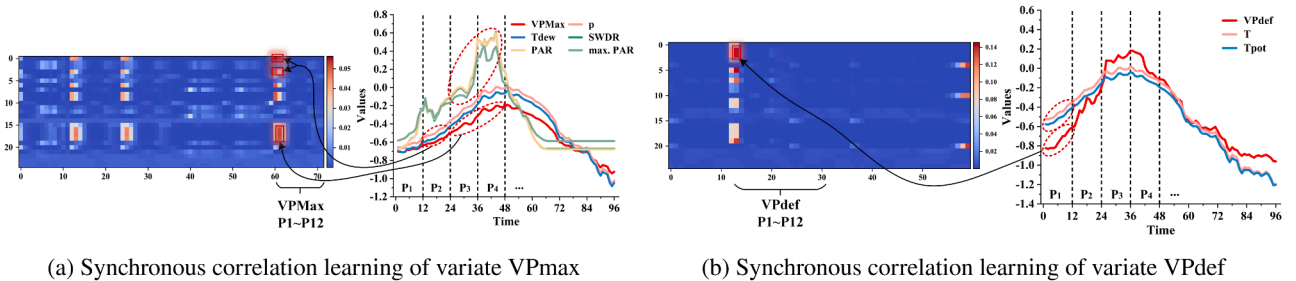


Fig. 7. Interpretable analysis of synchronous correlations on Weather datasets. (a) Synchronous correlation learning of variate VPmax. (b) Synchronous correlation learning of variate VPdef.

Table 6

Comparison of the performance of LLMs on long-term forecasting tasks. The look-back window L is 96 and the prediction horizons $H \in \{96, 192, 336, 720\}$. Details are the same as Table 2.

Models		AsynFormer Our		TimeCMA [2025]		LLM4TS [2025]		TEMPO [2024]		Time-LLM [2024]		UniTime [2024]		GPT4TS [2023]	
Metrics		MSE	MAE	MSE	MAE	MSE	MAE	MSE	MAE	MSE	MAE	MSE	MAE	MSE	MAE
ETTh2	96	0.173	0.251	0.173	0.258	0.176	0.255	0.185	0.267	0.193	0.280	0.183	0.266	0.190	0.275
	192	0.236	0.295	0.238	0.301	0.256	0.317	0.243	0.304	0.257	0.318	0.251	0.310	0.253	0.313
	336	0.296	0.333	0.297	0.338	0.312	0.345	0.309	0.345	0.317	0.353	0.319	0.351	0.321	0.360
	720	0.398	0.390	0.393	0.394	0.412	0.405	0.386	0.395	0.419	0.411	0.420	0.410	0.411	0.406
	AVG	0.276	0.317	0.275	0.323	0.289	0.331	0.281	0.328	0.296	0.340	0.293	0.334	0.294	0.339
ETTh2	96	0.284	0.333	0.286	0.336	0.307	0.345	0.317	0.352	0.295	0.345	0.296	0.345	0.312	0.360
	192	0.358	0.380	0.363	0.387	0.370	0.386	0.408	0.412	0.386	0.399	0.374	0.394	0.387	0.405
	336	0.413	0.420	0.406	0.421	0.414	0.432	0.419	0.452	0.419	0.429	0.415	0.427	0.424	0.437
	720	0.414	0.431	0.417	0.438	0.420	0.438	0.417	0.431	0.425	0.442	0.425	0.444	0.433	0.453
	AVG	0.367	0.391	0.372	0.397	0.378	0.400	0.390	0.412	0.381	0.404	0.378	0.403	0.389	0.414
Weather	96	0.156	0.195	0.167	0.211	0.204	0.262	0.222	0.265	0.198	0.235	0.171	0.214	0.203	0.244
	192	0.202	0.240	0.212	0.253	0.223	0.288	0.286	0.314	0.240	0.269	0.217	0.254	0.247	0.277
	336	0.260	0.282	0.270	0.292	0.381	0.362	0.337	0.349	0.295	0.308	0.274	0.293	0.297	0.311
	720	0.338	0.334	0.350	0.348	0.427	0.403	0.363	0.321	0.368	0.353	0.351	0.343	0.368	0.356
	AVG	0.239	0.263	0.250	0.276	0.309	0.329	0.302	0.312	0.275	0.291	0.253	0.276	0.279	0.297

Table 7

Comparison of AsynFormer's model complexity and training overhead. Experimental results are calculated using the optimal parameters for ETTh2 and ETTm2, using a 96-input-96-prediction setup.

Model	ETTh2					ETTh2				
Metrics	MSE	Training Time (s)	GPU peak (MB)	Params (MB)	FLOPs (MB)	MSE	Training Time (s)	GPU peak (MB)	Params (MB)	FLOPs (MB)
Crossformer	0.745	43.207	321.900	1.994	181.485	0.287	41.860	117.480	1.994	181.485
Pyraformer	0.645	41.980	170.600	0.543	68.765	0.435	39.990	86.640	0.543	68.765
TiDE	0.400	38.850	85.800	0.442	11.631	0.207	38.570	57.420	0.442	11.631
Autoformer	0.358	43.715	355.860	0.471	106.833	0.255	44.540	273.120	0.471	106.833
FEDformer	0.332	79.920	427.660	0.471	106.833	0.203	82.510	138.030	0.471	106.833
PatchTST	0.302	39.599	79.800	0.547	69.071	0.175	35.860	31.550	0.547	69.071
TimeXer	0.286	38.859	67.850	0.571	53.228	0.171	37.170	36.340	0.372	28.001
AsynFormer	0.284	35.250	116.110	0.451	54.664	0.173	35.140	67.120	0.451	54.664

ables 1 and 4 before and after training. This visualization demonstrates that AsynFormer's variable-wise proxy mechanism successfully recalibrates the relational weights between variables at different time steps to effectively capture and model the underlying asynchronous dependencies.

Synchronous Cross-Variate Visualization. Beyond its ability to model asynchronous cross-variable interactions, we also explore AsynFormer's capability to capture synchronous cross-variable dependencies through intuitive visual analysis. With an experimental configuration of a patch length $S = 8$ and a look-back window $L = 96$, we obtain patch-wise cross-variable attention weight heatmaps on the Weather dataset, as shown in Fig. 7. In Fig. 7 (a), we use the variable 5 (VPM_{max}) as the query. In the weight heatmap on the left, the x-axis represents 6 variables including VPM_{max}, and the y-axis represents all variables. The analysis shows that patches P3 and P4 of VPM_{max} have extremely high attention weights with variables 17 (SWDR), 18 (PAR), and 19 (Max PAR). Therefore, we visualize the corresponding variable series, from which it is clear that variables 5, 17, 18, and 19 exhibit very similar evolution patterns at patches 3 and 4. Furthermore, in Fig. 7 (b), we take variable VPM_{def} as the subject; the x-axis of the left weight heatmap represents 5 variables including VPM_{def}, and the y-axis represents all variables. Similarly, the corresponding variable series for the highlighted patches in the heatmap are also found to have similar patterns of change. This visual analysis demonstrates that AsynFormer accurately captures these synchronous dependencies and can effectively model cross-variable correlations within the same intervals.

5.6. Benchmarking against LLMs

Recently, the application of Large Language Models (LLMs) to MTSF has garnered significant attention, as their inherent ability to capture complex dependencies and semantic patterns offers a promising new paradigm [53,56]. To situate AsynFormer within this emerging landscape, we conduct a comprehensive benchmark against several state-of-the-art LLMs-based models. The experiment compares the performance on long-term forecasting tasks, where we fix the look-back window at $L = 96$ and evaluate all models across multiple prediction horizons, specifically $H \in \{96, 192, 336, 720\}$. The empirical results, detailed in Table 6, unequivocally demonstrate the superior performance of AsynFormer. Across all evaluated datasets (ETTh2, ETTh2, and Weather), AsynFormer consistently maintains excellent prediction performance, with MAEs of 0.317, 0.391, and 0.263, respectively, representing average improvements of 1.81%, 1.53%, and 5.04% compared to TimeCMA (0.323, 0.397, 0.276). This sustained out-performance highlights that our proposed dual-stream branch architecture can effectively capture asynchronous cross-variable dynamics within time series, achieving results comparable to LLMs-based methods that rely on massive parameter counts or the integration of auxiliary textual information.

5.7. Computational efficiency analysis

To systematically evaluate the trade-off between model performance and computational resource consumption, we conducted a comprehensive efficiency analysis on the ETTh2 and ETTm2. All experiments were performed under a unified '96-input-96-output' setting to ensure a fair comparison. We benchmarked AsynFormer against leading long-term

Table 8

Complete parameter sensitivity data for AsynFormer. S is the patch length, and α is the loss balance parameter. The look-back window L is uniformly set to 96.

S	Metrics	Weather		ETTh1		α	Metrics	Weather		ETTh1	
		MSE	MAE	MSE	MAE			MSE	MAE	MSE	MAE
4	96	0.158	0.198	0.384	0.390	0.15	96	0.156	0.195	0.380	0.391
	192	0.207	0.245	0.442	0.424		192	0.202	0.240	0.429	0.419
	336	0.264	0.284	0.475	0.440		336	0.260	0.282	0.475	0.440
	720	0.338	0.334	0.466	0.460		720	0.338	0.334	0.466	0.460
	AVG	0.242	0.265	0.442	0.429		AVG	0.239	0.263	0.438	0.428
8	96	0.157	0.195	0.380	0.391	0.20	96	0.160	0.199	0.381	0.390
	192	0.204	0.242	0.429	0.419		192	0.206	0.241	0.441	0.422
	336	0.262	0.284	0.475	0.440		336	0.262	0.285	0.475	0.441
	720	0.340	0.335	0.466	0.460		720	0.340	0.337	0.473	0.465
	AVG	0.241	0.264	0.438	0.428		AVG	0.242	0.266	0.443	0.430
12	96	0.156	0.195	0.382	0.393	0.25	96	0.157	0.198	0.391	0.391
	192	0.202	0.240	0.430	0.419		192	0.205	0.241	0.431	0.420
	336	0.260	0.282	0.478	0.444		336	0.264	0.285	0.477	0.442
	720	0.338	0.334	0.469	0.477		720	0.341	0.336	0.472	0.463
	AVG	0.239	0.263	0.440	0.433		AVG	0.242	0.265	0.443	0.429
16	96	0.158	0.198	0.383	0.395	0.30	96	0.159	0.198	0.382	0.391
	192	0.207	0.242	0.437	0.423		192	0.208	0.242	0.437	0.419
	336	0.262	0.283	0.476	0.450		336	0.264	0.286	0.478	0.442
	720	0.343	0.337	0.473	0.478		720	0.343	0.338	0.465	0.463
	AVG	0.243	0.265	0.442	0.437		AVG	0.244	0.266	0.441	0.429
32	96	0.159	0.198	0.384	0.394	0.35	96	0.158	0.198	0.384	0.392
	192	0.210	0.246	0.437	0.424		192	0.206	0.242	0.436	0.419
	336	0.263	0.284	0.475	0.467		336	0.262	0.285	0.479	0.444
	720	0.347	0.340	0.474	0.467		720	0.342	0.337	0.466	0.460
	AVG	0.245	0.267	0.443	0.438		AVG	0.242	0.266	0.441	0.429

Table 9

Sensitivity analysis of the number of Context Proxies (G_c) per variable on Weather and ETTh1 datasets.

Number of G_c	Metrics	Weather		ETTh1	
		MSE	MAE	MSE	MAE
$G_c = 1$	96	0.156	0.195	0.380	0.391
	192	0.202	0.240	0.429	0.419
	336	0.260	0.282	0.475	0.440
	720	0.338	0.334	0.466	0.460
	AVG	0.239	0.263	0.438	0.428
$G_c = 2$	96	0.157	0.197	0.382	0.393
	192	0.204	0.243	0.441	0.417
	336	0.259	0.283	0.477	0.442
	720	0.342	0.336	0.469	0.462
	AVG	0.241	0.265	0.442	0.429
$G_c = 3$	96	0.155	0.197	0.383	0.394
	192	0.204	0.242	0.443	0.421
	336	0.262	0.283	0.477	0.441
	720	0.339	0.335	0.468	0.462
	AVG	0.240	0.264	0.443	0.430

forecasting models across four key metrics: training speed (s/epoch), GPU peak memory usage (MB), parameter count (MB), and computational complexity (FLOPs). The detailed statistics are presented in Table 7, and a visual comparison of the trade-offs on ETTh2 is illustrated in Fig. 8.

Parameter Efficiency and Training Speed. AsynFormer demonstrates remarkable efficiency in model size and convergence speed. As shown in Table 7, our model requires only 0.45 MB of parameters. This represents a reduction of approximately 17.5% and 21.0% compared to the current state-of-the-art models PatchTST (0.547 MB) and TimeXer

Table 10

Optimal hyper-parameter settings of AsynFormer on all datasets.

Dataset	Batch Size	S	α	Learning Rate	E-layers
ETTh1	4	8	0.15	1e-4	2
ETTh2	16	12	0.35	1e-4	1
ETTm1	64	8	0.15	1e-4	1
ETTm2	64	12	0.10	2e-4	2
Exchange	16	12	0.20	1e-4	1
Weather	16	12	0.15	1e-4	4
Traffic	8	32	0.15	3e-4	3
PeMSO3	32	8	0.20	1e-4	5
PeMSO4	8	8	0.15	2e-4	2
PeMSO7	4	8	0.20	2e-4	3
PeMSO8	4	12	0.20	1e-4	5

(0.571 MB), respectively. This lightweight design translates directly into training efficiency: AsynFormer completes one training epoch in just 35.25 s, outperforming PatchTST by over 10% and the complex Crossformer by nearly 20%. Unlike Pyraformer and Crossformer, which suffer from bloated parameter spaces without proportional accuracy gains, AsynFormer achieves the lowest MSE with a significantly more compact architecture.

Complexity vs. Accuracy Trade-off. While minimizing computational overhead is crucial, it must not come at the expense of predictive power.

- *Comparison with MLP-based models:* Although the MLP-based TiDE model achieves the lowest theoretical FLOPs (11.631 MB), it struggles to capture complex temporal dependencies, resulting in a much higher MSE (0.400). In contrast, AsynFormer reduces the prediction

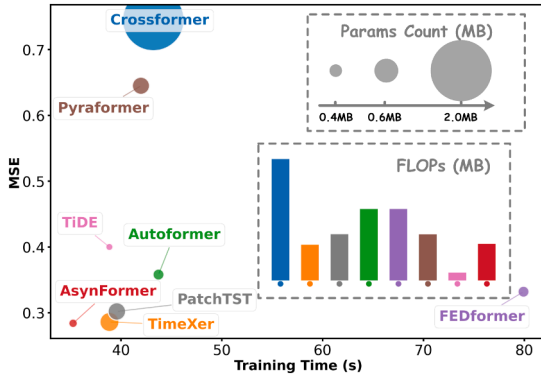


Fig. 8. Computational efficiency on ETTh2 dataset.

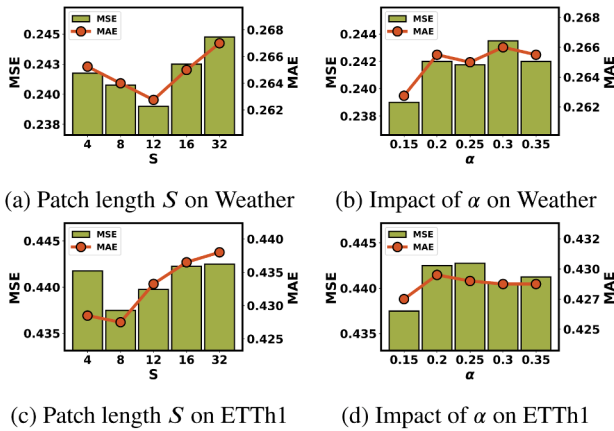


Fig. 9. Hyperparameter sensitivity analysis for the Weather and ETTh1 datasets with a look-back window of $L = 96$. Results are averaged over prediction lengths $H \in \{96, 192, 336, 720\}$. Detailed data is shown in Table 8. (a) Patch length S on Weather. (b) Impact of α on Weather. (c) Patch length S on ETTh1. (d) Impact of α on ETTh1.

error by 29% compared to TiDE. While our FLOPs are higher than TiDE, this moderate increase is a justifiable investment for the substantial leap in accuracy.

- **Comparison with Transformer variants:** Compared to standard Transformers like Crossformer (181.485 MB FLOPs), AsynFormer reduces computational complexity by approximately 70% (54.664 MB FLOPs) while achieving significantly better accuracy (MSE 0.284 vs. 0.745).

These results strongly indicate that AsynFormer achieves the optimal balance between predictive accuracy and computational efficiency. The superior performance is attributed to our core asynchronous attention mechanism, which effectively captures fine-grained temporal interactions without incurring the heavy computational burden of full-attention maps or the information loss of over-simplified linear projections. In summary, AsynFormer not only achieves state-of-the-art prediction performance but is also highly competitive in terms of both parameter efficiency and runtime.

5.8. Effect of hyper-parameters

We conduct a thorough analysis to investigate the sensitivity of the AsynFormer model to its key hyperparameters. This section focuses on the impact of the patch length S , the loss balancing coefficient α and the number of G_c . In our experiments, we configure a fixed look-back window of $L = 96$ and evaluate the model across multiple prediction horizons $H \in \{96, 192, 336, 720\}$. We report the average MSE and MAE performance on the Weather and ETTh1 benchmark datasets. Fig. 9 in-

tuitively illustrates the performance trends as the hyperparameters vary, while Table 8 provides the detailed numerical results. Based on this data, we draw the following conclusions:

- **Sensitivity to patch length S :** As shown in Fig. 9 (a), (c), as the patch length S increases from 4 to 32, the model's MSE/MAE performance metrics exhibit a trend of first decreasing and then increasing, achieving optimal performance at a specific value. Specifically, the model performs best with $S = 12$ on the Weather dataset, whereas the optimal patch length is $S = 8$ on the ETTh1 dataset. This disparity suggests that different datasets possess distinct, inherent local temporal patterns and cross-variable correlations. Therefore, tailoring the patch length to the specific dataset is crucial for the model to achieve its peak prediction accuracy.
- **Sensitivity to loss balance α :** Fig. 9 (b), (d) illustrates the impact of the loss balancing coefficient α on model performance. It is observed that the model achieves peak performance on both datasets when $\alpha = 0.15$. As the value of α further increases, the MSE/MAE metrics on both datasets deteriorate to varying degrees, and the trends of performance degradation differ between them. This phenomenon indicates that the frequency-domain loss provides an effective supervisory signal to ensure the consistency of features in the frequency domain. However, it should not act as a dominant signal but rather as an auxiliary term to the primary MAE loss. The model can only achieve its optimal predictive performance through this synergistic effect.
- **Sensitivity to the number of G_c :** To investigate whether a single proxy per variable creates an information bottleneck, we conducted a sensitivity analysis by varying the number of context proxies (G_c) from 1 to 3, as detailed in Table 9. It is observed that increasing the number of proxies does not yield significant performance gains. Instead, the performance remains stable or exhibits slight degradation as the proxy count increases. For instance, on the ETTh1 dataset, the average MSE/MAE marginally increases from 0.438/0.428 ($G_c = 1$) to 0.443/0.430 ($G_c = 3$). This phenomenon indicates that a single high-dimensional proxy vector is sufficiently expressive to aggregate the core temporal dynamics of an individual time series. Increasing the number of proxies tends to introduce redundancy and potential overfitting risks rather than providing additional information gain. Consequently, we adopt $G_c = 1$ as the optimal setting to achieve the best trade-off between computational efficiency and predictive accuracy.

Finally, the optimal hyperparameter configurations for our proposed AsynFormer model on the various benchmark datasets are summarized in Table 10.

6. Conclusion

In this paper, we introduced **AsynFormer**, a novel unified Transformer-based framework for MTSF designed explicitly to address asynchronous cross-variate interactions and fine-grained temporal dynamics. Distinct from existing synchronous attention approaches, AsynFormer concurrently captures asynchronous cross-variate dependencies and local intra-series dynamics through its dual-stream architecture. To ensure computational scalability, we proposed a variable-wise proxy module leveraging lightweight, learnable proxy tokens to aggregate global context across variables, significantly reducing complexity. A comprehensive empirical evaluation against a range of state-of-the-art models on eleven widely-used datasets confirms the superiority of AsynFormer. The model significantly outperforms competing baselines in both short- and long-term forecasting tasks while maintaining high computational efficiency.

CRedit authorship contribution statement

Peng He: Visualization, Resources, Project administration, Methodology; **Yanglei Gan:** Writing – original draft, Project administration; **Yuxiang Liu:** Validation, Supervision; **Run Lin:** Project administration; **Guanyu Zhou:** Visualization, Software; **Jun Xie:** Investigation, Funding acquisition, Formal analysis; **Yao Liu:** Supervision, Conceptualization; **Qiao Liu:** Methodology, Investigation, Funding acquisition.

Data availability

Data will be made available on request.

Declaration of competing interest

The authors declare that they have no known competing financial interests or personal relationships that could have appeared to influence the work reported in this paper.

Acknowledgements

We would like to thank the anonymous reviewers for their valuable discussion and constructive feedback. This work is supported by National Natural Science Foundation of China (U22B2061), National Key R&D Program of China (2022YFB4300603), and Natural Science Foundation of Sichuan, China (2024NSFSC0496).

References

- [1] L.-J. Cao, F.E.H. Tay, Support vector machine with adaptive parameters in financial time series forecasting, *IEEE Trans. Neural Netw.* 14 (6) (2003) 1506–1518.
- [2] O.B. Sezer, M.U. Gudelek, A.M. Ozbayoglu, Financial time series forecasting with deep learning: a systematic literature review: 2005–2019, *Appl. Soft Comput.* 90 (2020) 106181.
- [3] M. Lippi, M. Bertini, P. Frasconi, Short-term traffic flow forecasting: an experimental comparison of time-series analysis and supervised learning, *IEEE Trans. Intell. Transp. Syst.* 14 (2) (2013) 871–882.
- [4] R.-G. Cirstea, B. Yang, C. Guo, T. Kieu, S. Pan, Towards spatio-temporal aware traffic time series forecasting, in: 2022 IEEE 38th International Conference on Data Engineering, IEEE, 2022, pp. 2900–2913.
- [5] Z. Karevan, J.A.K. Suykens, Transductive LSTM for time-series prediction: an application to weather forecasting, *Neural Netw.* 125 (2020) 1–9.
- [6] Y. Chen, K. Yang, Z. An, B. Holder, L. Paloutzian, K.M. Bali, W. Du, MARLP: time-series forecasting control for agricultural managed aquifer recharge, in: Proceedings of the 30th ACM SIGIR Conference on Knowledge Discovery and Data Mining, 2024, pp. 4862–4872.
- [7] C. Deb, F. Zhang, J. Yang, S.E. Lee, K.W. Shah, A review on time series forecasting techniques for building energy consumption, *Renewable Sustainable Energy Rev.* 74 (2017) 902–924.
- [8] N. Maleki, O. Lundström, A. Musaddiq, J. Jeansson, T. Olsson, F. Ahlgren, Future energy insights: time-series and deep learning models for city load forecasting, *Appl. Energy* 374 (2024) 124067.
- [9] Y. Zhang, J. Yan, Crossformer: transformer utilizing cross-dimension dependency for multivariate time series forecasting, in: The Eleventh International Conference on Learning Representations, 2023, pp. 1–21.
- [10] Y. Liu, T. Hu, H. Zhang, H. Wu, S. Wang, L. Ma, M. Long, iTransformer: inverted transformers are effective for time series forecasting, in: The Twelfth International Conference on Learning Representations, 2024, pp. 1–25.
- [11] G. Yu, J. Zou, X. Hu, A.I. Aviles-Rivero, J. Qin, S. Wang, Revitalizing multivariate time series forecasting: learnable decomposition with inter-series dependencies and intra-series variations modeling, in: International Conference on Machine Learning, PMLR, 2024, pp. 57818–57841.
- [12] W. Cai, Y. Liang, X. Liu, J. Feng, Y. Wu, MSGNet: learning multi-scale inter-series correlations for multivariate time series forecasting, in: Proceedings of the Association for the Advancement of Artificial Intelligence Conference on Artificial Intelligence, 38, 2024, pp. 11141–11149.
- [13] Y. Nie, N.H. Nguyen, P. Sinthong, J. Kalagnanam, A time series is worth 64 words: long-term forecasting with transformers, in: The Eleventh International Conference on Learning Representations, 2023.
- [14] P. Chen, Y. ZHANG, Y. Cheng, Y. Shu, Y. Wang, Q. Wen, B. Yang, C. Guo, Pathformer: multi-scale transformers with adaptive pathways for time series forecasting, in: The Twelfth International Conference on Learning Representations, 2024, pp. 1–19.
- [15] Q. Huang, L. Shen, R. Zhang, J. Cheng, S. Ding, Z. Zhou, Y. Wang, HDMixer: hierarchical dependency with extendable patch for multivariate time series forecasting, in: Proceedings of the Association for the Advancement of Artificial Intelligence Conference on Artificial Intelligence, 38, 2024, pp. 12608–12616.
- [16] W. Zhang, C. Yin, H. Liu, X. Zhou, H. Xiong, Irregular multivariate time series forecasting: a transformable patching graph neural networks approach, in: Forty-first International Conference on Machine Learning, 2024, pp. 1–18.
- [17] S. Xu, A. Khandelwal, A. Renganathan, V. Kumar, Message propagation through time: an algorithm for sequence dependency retention in time series modeling, in: Proceedings of the 2024 SIAM International Conference on Data Mining, SIAM, 2024, pp. 307–315.
- [18] A. Behrouz, M. Santacatterina, R. Zabih, Chimera: effectively modeling multivariate time series with 2-dimensional state space models, *Adv. Neural Inf. Process. Syst.* 37 (2024) 119886–119918.
- [19] H. Khadem, H. Nemat, J. Elliott, M. Benaissa, Blood glucose level time series forecasting: nested deep ensemble learning lag fusion, *Bioengineering* 10 (4) (2023) 487.
- [20] A. Vaswani, N. Shazeer, N. Parmar, J. Uszkoreit, L. Jones, A.N. Gomez, L. Kaiser, I. Polosukhin, Attention is all you need, in: Proceedings of the 31st International Conference on Neural Information Processing Systems, 2017, pp. 6000–6010.
- [21] A. Dosovitskiy, L. Beyer, A. Kolesnikov, D. Weissenborn, X. Zhai, T. Unterthiner, M. Dehghani, M. Minderer, G. Heigold, S. Gelly, J. Uszkoreit, N. Houlsby, An image is worth 16x16 words: transformers for image recognition at scale, in: International Conference on Learning Representations, 2021, pp. 1–22.
- [22] M. Liu, A. Zeng, M. Chen, Z. Xu, Q. Lai, L. Ma, Q. Xu, SciNet: time series modeling and forecasting with sample convolution and interaction, *Adv. Neural Inf. Process. Syst.* 35 (2022) 5816–5828.
- [23] C. Liu, X. Wang, X. Jiang, Y. Sui, S. Cao, C. Qi, H. Liu, A parallel and multi-scale probabilistic temporal convolutional neural networks for forecasting the key monitoring parameters of gas turbine, *Eng. Appl. Artif. Intell.* 138 (2024) 109400.
- [24] S. Siami-Namini, N. Tavakoli, A.S. Namin, A comparison of ARIMA and LSTM in forecasting time series, in: 2018 17th IEEE International Conference on Machine Learning and Applications, Ieee, 2018, pp. 1394–1401.
- [25] Y. Tang, F. Yu, W. Pedrycz, X. Yang, J. Wang, S. Liu, Building trend fuzzy granulation-based LSTM recurrent neural network for long-term time-series forecasting, *IEEE Trans. Fuzzy Syst.* 30 (6) (2021) 1599–1613.
- [26] H. Wu, J. Xu, J. Wang, M. Long, Autoformer: decomposition transformers with auto-correlation for long-term series forecasting, *Adv. Neural Inf. Process. Syst.* 34 (2021) 22419–22430.
- [27] H. Zhou, S. Zhang, J. Peng, S. Zhang, J. Li, H. Xiong, W. Zhang, Informer: beyond efficient transformer for long sequence time-series forecasting, in: Proceedings of the Association for the Advancement of Artificial Intelligence Conference on Artificial Intelligence, 35, 2021, pp. 11106–11115.
- [28] A. Kazemnejad, I. Padhi, K. Natesan Ramamurthy, P. Das, S. Reddy, The impact of positional encoding on length generalization in transformers, *Adv. Neural Inf. Process. Syst.* 36 (2023) 24892–24928.
- [29] N.M. Fomani, C.W. Tan, G.I. Webb, M. Salehi, Improving position encoding of transformers for multivariate time series classification, *Data Min. Knowl. Discov.* 38 (1) (2024) 22–48.
- [30] N. Kitaev, L. Kaiser, A. Levskaya, Reformer: the efficient transformer, in: International Conference on Learning Representations, 2020, pp. 1–12.
- [31] T. Zhou, Z. Ma, Q. Wen, X. Wang, L. Sun, R. Jin, Fedformer: frequency enhanced decomposed transformer for long-term series forecasting, in: International Conference on Machine Learning, PMLR, 2022, pp. 27268–27286.
- [32] S. Liu, H. Yu, C. Liao, J. Li, W. Lin, A.X. Liu, S. Dustdar, Pyraformer: low-complexity pyramid attention for long-range time series modeling and forecasting, in: International Conference on Learning Representations, 2022, pp. 1–20.
- [33] Z. Wang, Z. Sun, Y. Bian, H. Mo, D. Dong, Learning hierarchical time-frequency representation for long-term time series forecasting, *Inf. Process. Manage.* 63 (2) (2026) 104358.
- [34] H. Wang, L. Pan, Y. Shen, Z. Chen, D. Yang, Y. Yang, S. Zhang, X. Liu, H. Li, D. Tao, FedDF: learning to forecast in the frequency domain, in: The Thirteenth International Conference on Learning Representations, 2025, pp. 1–33.
- [35] H. Lee, Z. Zeng, Z. Qiu, W. Zhu, R. Xiao, CSCAD: modeling cross-scale sequence correlations for multivariate time series anomaly detection, *Inf. Process. Manage.* 63 (1) (2026) 104315.
- [36] T. Xiong, Y. Bao, Z. Hu, Interval forecasting of electricity demand: a novel bivariate EMD-based support vector regression modeling framework, *Int. J. Electr. Power Energy Syst.* 63 (2014) 353–362.
- [37] K. He, Y. Chen, G.K.F. Tso, Price forecasting in the precious metal market: a multivariate EMD denoising approach, *Resour. Policy* 54 (2017) 9–24.
- [38] W.-c. Wang, Q. Cheng, K.-w. Chau, H. Hu, H.-f. Zang, D.-m. Xu, An enhanced monthly runoff time series prediction using extreme learning machine optimized by salp swarm algorithm based on time varying filtering based empirical mode decomposition, *J. Hydrol.* 620 (2023) 129460.
- [39] Y. Yao, Z.-y. Zhang, Y. Zhao, Stock index forecasting based on multivariate empirical mode decomposition and temporal convolutional networks, *Appl. Soft Comput.* 142 (2023) 110356.
- [40] X. Cai, D. Li, J. Zhang, Z. Wu, MA-EMD: aligned empirical decomposition for multivariate time-series forecasting, *Expert Syst. Appl.* 267 (2025) 126080.
- [41] A.-O. Boudraa, J.-C. Cexus, EMD-based signal filtering, *IEEE Trans. Instrum. Meas.* 56 (6) (2007) 2196–2202.
- [42] X. Wang, T. Zhou, Q. Wen, J. Gao, B. Ding, R. Jin, CARD: channel aligned robust blend transformer for time series forecasting, in: The Twelfth International Conference on Learning Representations, 2024, pp. 1–39.
- [43] Y. Wang, H. Wu, J. Dong, G. Qin, H. Zhang, Y. Liu, Y. Qiu, J. Wang, M. Long, TimeXer: empowering transformers for time series forecasting with exogenous variables, *Adv. Neural Inf. Process. Syst.* 37 (2024) 469–498.

- [44] L. Zhao, Y. Shen, Rethinking channel dependence for multivariate time series forecasting: learning from leading indicators, in: The Twelfth International Conference on Learning Representations, 2024, pp. 1–19.
- [45] Y. Wang, Y. Xu, J. Yang, M. Wu, X. Li, L. Xie, Z. Chen, Fully-connected spatial-temporal graph for multivariate time-series data, in: Proceedings of the Association for the Advancement of Artificial Intelligence Conference on Artificial Intelligence, 38, 2024, pp. 15715–15724.
- [46] D. Sun, J. Qin, Z. Zhang, X. Qin, H. Zhang, MRLCD-a: lag-aware alignment for multivariate time series forecasting in multiple scenarios, *Inf. Process. Manage.* 62 (5) (2025) 104191.
- [47] G. Lai, W.-C. Chang, Y. Yang, H. Liu, Modeling long-and short-term temporal patterns with deep neural networks, in: The 41st International ACM SIGIR Conference on Research & Development in Information Retrieval, 2018, pp. 95–104.
- [48] H. Wu, Revisiting attention for multivariate time series forecasting, in: Proceedings of the Association for the Advancement of Artificial Intelligence Conference on Artificial Intelligence, 39, 2025, pp. 21528–21535.
- [49] Z. Xu, A. Zeng, Q. Xu, FITS: modeling time series with \$10k\$ parameters, in: The Twelfth International Conference on Learning Representations, 2024, pp. 1–23.
- [50] K. Yi, J. Fei, Q. Zhang, H. He, S. Hao, D. Lian, W. Fan, FilterNet: harnessing frequency filters for time series forecasting, *Adv. Neural Inf. Process. Syst.* 37 (2024) 55115–55140.
- [51] K. Yi, Q. Zhang, W. Fan, S. Wang, P. Wang, H. He, N. An, D. Lian, L. Cao, Z. Niu, Frequency-domain MLPs are more effective learners in time series forecasting, *Adv. Neural Inf. Process. Syst.* 36 (2023) 76656–76679.
- [52] A. Das, W. Kong, A. Leach, S.K. Mathur, S. Rajat, R. Yu, Long-term forecasting with TiDE: time-series dense encoder, *Trans. Mach. Learn. Res.* (2023) 1–21.
- [53] C. Liu, Q. Xu, H. Miao, S. Yang, L. Zhang, C. Long, Z. Li, R. Zhao, TimeCMA: towards LLM-empowered multivariate time series forecasting via cross-modality alignment, in: Proceedings of the Association for the Advancement of Artificial Intelligence Conference on Artificial Intelligence, 39, 2025, pp. 18780–18788.
- [54] C. Chang, W.-Y. Wang, W.-C. Peng, T.-F. Chen, LLM4TS: aligning pre-trained LLMs as data-efficient time-series forecasters, *ACM Trans. Intell. Syst. Technol.* 16 (3) (2025) 1–20.
- [55] D. Cao, F. Jia, S.O. Arik, T. Pfister, Y. Zheng, W. Ye, Y. Liu, TEMPO: prompt-based generative pre-trained transformer for time series forecasting, in: The Twelfth International Conference on Learning Representations, 2024, pp. 1–33.
- [56] M. Jin, S. Wang, L. Ma, Z. Chu, J.Y. Zhang, X. Shi, P.-Y. Chen, Y. Liang, Y.-F. Li, S. Pan, Q. Wen, Time-LLM: time series forecasting by reprogramming large language models, in: International Conference on Learning Representations, 2024, pp. 1–24.
- [57] X. Liu, J. Hu, Y. Li, S. Diao, Y. Liang, B. Hooi, R. Zimmermann, UniTime: a language-empowered unified model for cross-domain time series forecasting, in: Proceedings of the ACM Web Conference 2024, 2024, pp. 4095–4106.
- [58] T. Zhou, P. Niu, L. Sun, R. Jin, et al., One fits all: power general time series analysis by pretrained LM, *Adv. Neural Inf. Process. Syst.* 36 (2023) 43322–43355.
- [59] Y. Hu, P. Liu, P. Zhu, D. Cheng, T. Dai, Adaptive multi-scale decomposition framework for time series forecasting, in: Proceedings of the Association for the Advancement of Artificial Intelligence Conference on Artificial Intelligence, 39, 2025, pp. 17359–17367.
- [60] H. Wu, T. Hu, Y. Liu, H. Zhou, J. Wang, M. Long, TimesNet: temporal 2D-variation modeling for general time series analysis, in: International Conference on Learning Representations, 2023, pp. 1–23.
- [61] Z. Li, Z. Rao, L. Pan, Z. Xu, MTS-mixers: multivariate time series forecasting via factorized temporal and channel mixing, in: 2025 International Joint Conference on Neural Networks (IJCNN) (2025) 1–8.
- [62] A. Zeng, M. Chen, L. Zhang, Q. Xu, Are transformers effective for time series forecasting?, in: Proceedings of the Association for the Advancement of Artificial Intelligence Conference on Artificial Intelligence, 37, 2023, pp. 11121–11128.
- [63] Z. Li, S. Qi, Y. Li, Z. Xu, Revisiting long-term time series forecasting: an investigation on affine mapping, *CoRR abs/2305.10721* (2023) 1–12.
- [64] D.P. Kingma, J. Ba, Adam: a method for stochastic optimization, *CoRR abs/1412.6980* (2014) 1–15.
- [65] H. Hewamalage, K. Ackermann, C. Bergmeir, Forecast evaluation for data scientists: common pitfalls and best practices, *Data Min. Knowl. Discov.* 37 (2) (2023) 788–832.